

Documentation technique

Table des matières

1. Introduction	5
1.1 Objet du document.....	5
1.2 Public vise.....	5
1.3 Perimetre du document.....	5
2. Contexte et objectifs	6
2.1 Probleme traite	6
2.2 Objectifs du projet.....	6
2.3 Positionnement	7
3. Perimetre fonctionnel realise	7
3.1 Back-office.....	7
3.2 Espace client.....	8
3.3 Roles applicatifs	8
3.4 Modules metier principaux	9
4. Stack technique et justification des choix.....	9
4.1 Tableau de la stack.....	9
4.2 Backend : PHP 8.2 et Symfony 6.4	10
4.3 Frontend : React, TypeScript et Vite.....	11
4.4 Base de donnees : MySQL 8.....	11
4.5 Docker et verrouillage des versions	11
5. Architecture generale.....	12
5.1 Vue d'ensemble	12
5.2 Organisation du depot.....	13
5.3 Separation des responsabilites.....	13
5.4 Diagramme de cas d'utilisation	16
6. Architecture backend.....	17
6.1 Structure principale	17
6.2 Controllers principaux	17
6.3 Services metier principaux.....	18

6.4 Gestion des erreurs.....	18
7. Architecture frontend	18
7.1 Structure principale	19
7.2 Espaces applicatifs.....	19
7.3 Pages structurantes.....	19
7.4 Principes UI.....	20
8. Architecture de la base de donnees.....	20
8.1 Vue d'ensemble	20
8.2 Utilisateurs et profils	23
8.3 Catalogue et stock	23
8.4 Vente et paiement	24
8.5 Rendez-vous et planning.....	24
8.6 CRM et fidelite.....	25
8.7 E-commerce.....	25
9. Conception de securite	25
9.1 Authentification	25
9.2 Autorisation.....	26
9.3 Gestion des mots de passe.....	27
9.4 Secrets et variables sensibles.....	27
9.5 Paiement et webhooks.....	28
9.6 Donnees exposees	28
10. API et contrats d'echange	28
10.1 Conventions generales	28
10.2 Routes de base	29
10.3 Catalogue et stock.....	29
10.4 E-commerce.....	29
10.5 Fidelite et bons cadeaux.....	30
10.6 Reservation produit.....	30
10.7 Documentation OpenAPI	30
11. Outils de vente et de reservation.....	31
11.1 Caisse / POS	31
11.2 Planning.....	32
11.3 E-commerce	33

12. CRM, fidelite et relation client.....	34
12.1 Points de fidelite	34
12.2 Membership et cartes de visites	34
12.3 Campagnes.....	35
12.4 Bons cadeaux.....	35
13. Paiement, notifications et integrations externes	35
13.1 Stripe.....	35
13.2 Emails	35
13.3 SMS	36
14. Infrastructure et deploiement local.....	36
14.1 Stack Docker	36
14.2 Ports utiles.....	37
14.3 Demarrage	38
15. Prise en main developpeur	38
15.1 Installation pas a pas.....	38
15.2 Comptes de test locaux.....	39
15.3 Commandes de maintenance.....	40
15.4 Variables d'environnement critiques.....	40
16. Tests et strategie de validation.....	40
16.1 Strategie globale.....	40
16.2 Backend	41
16.3 Frontend.....	41
16.4 Captures documentaires.....	42
17. Difficultes connues et solutions	42
17.1 Versions locales plus recentes.....	42
17.2 Ports deja utilises.....	42
17.3 Emails non recus	42
17.4 Donnees E2E visibles en prod locale	43
17.5 Taille des chunks frontend.....	43
18. Limites et perspectives	43
18.1 Limites actuelles.....	43
18.2 Perspectives fonctionnelles.....	44
18.3 Perspectives techniques	44

19. Annexes techniques.....	44
19.1 Documentation disponible.....	44
19.2 Scripts utiles	44
20. Conclusion.....	45

1. Introduction

1.1 Objet du document

Ce document présente les choix techniques, l'architecture et les principes d'implémentation de Procuratio.

Il est destiné aux professionnels du secteur informatique qui doivent comprendre, évaluer, maintenir ou faire évoluer le projet.

Il permet de:

- comprendre les choix de langage, framework et infrastructure
- expliquer l'organisation frontend, backend et base de données
- décrire les règles de sécurité et les contrats principaux
- documenter les modules de vente, réservation, CRM et e-commerce
- faciliter la reprise du projet par une équipe technique

1.2 Public vise

Cette documentation technique s'adresse principalement à :

- un jury technique
- un développeur reprenant le projet
- un administrateur devant lancer la stack
- un évaluateur souhaitant comprendre les choix d'implémentation

La documentation utilisateur doit rester séparée : elle explique les fonctionnalités aux clients, employés et managers sans entrer dans les détails de code.

1.3 Périmètre du document

Le document couvre :

- l'architecture applicative
- le model de données
- la securite
- les flux métier critiques
- les integrations externes
- les commandes utiles
- les limites connues
- les pistes d'amélioration

2. Contexte et objectifs

2.1 Problème traité

Un institut de beauté ou un salon de coiffure manipule plusieurs flux souvent disperses entre plusieurs outils :

- caisse et ventes physiques
- catalogue produits et prestations
- planning des rendez-vous
- fiches clients
- fidelite et cartes de visites
- campagnes CRM
- bons cadeaux
- commandes e-commerce
- preparation et suivi de stock

Procuratio centralise ces flux dans une même application afin de réduire les doubles saisies et de fournir une vision cohérente de l'activité.

2.2 Objectifs du projet

Les objectifs fonctionnels sont :

- proposer un back-office pour les opérations magasin
- proposer un espace client autonome
- gérer la vente physique, la réservation et l'e-commerce
- lier les ventes, rendez-vous et actions CRM aux fiches clients
- fournir une base de démonstration stable et compréhensible

Les objectifs techniques sont :

- fournir une API claire et maintenable
- garder une séparation stricte entre frontend, backend et base de données
- sécuriser les actions selon les rôles
- rendre la stack reproductible avec Docker
- verrouiller les versions importantes pour limiter les écarts de poste
- conserver des tests utiles sur les parcours critiques

2.3 Positionnement

Procuratio est pensé comme un ERP/CRM métier pour salons et instituts.

Le projet ne cherche pas une architecture inutilement complexe : il privilégie des choix robustes, lisibles et adaptés à un logiciel de gestion.

3. Périmètre fonctionnel réalisé

3.1 Back-office

Le back-office regroupe les fonctionnalités opérationnelles :

- dashboard d'activités

- gestion des produits
- gestion des prestations
- caisse / POS
- planning
- fiches clients
- gestion des employes
- gestion des managers
- gestion des boutiques
- CRM
- warehouse / preparation
- profil utilisateur

3.2 Espace client

L'espace client permet :

- consultation du catalogue
- gestion du panier
- passage de commande
- consultation des commandes
- annulation d'une commande non expédiée
- consultation des rendez-vous
- consultation des bons cadeaux disponibles
- modification du mot de passe
- modification de certaines preferences personnelles

3.3 Roles applicatifs

Les roles principaux sont :

- `ROLE\_CUSTOMER` : client final
- `ROLE\_EMPLOYEE` : employe operationnel
- `ROLE\_ADMIN` : manager ou administrateur

Un employé gère les opérations quotidiennes.

Un manager dispose d'un périmètre plus large : boutiques, employés, managers, horaires, données de référence et administration.

3.4 Modules métier principaux

- catalogue produits
- catalogue services
- stock
- caisse
- rendez-vous
- disponibilites employes
- horaires boutique
- CRM
- fidelite
- bons cadeaux
- e-commerce
- notifications
- statistiques

4. Stack technique et justification des choix

4.1 Tableau de la stack

Couche	Technologie	Version de reference	Role dans le projet
Language backend	PHP	8.2	Exécution de l'API et services métier
Framework backend	Symfony	6.4	Routage, sécurité, injection de dépendances, structure applicative
ORM	Doctrine ORM	version verrouillée dans `composer.lock`	Mapping entités, repositories et migrations
Base de données	MySQL	8.0	Persistance

			relationnelle des ventes, rendez-vous, stock et CRM
Language frontend	TypeScript	version verrouillée dans `package-lock.json`	Typage des écrans, payloads et réponses API
Framework frontend	React	18	Interfaces back-office, client, POS, planning et CRM
Build frontend	Vite	version verrouillée dans `package-lock.json`	Build rapide et bundling de l'application web
Authentication	JWT	configuration Symfony	Sessions stateless et contrôle d'accès API
Paielement	Stripe	API externe	Paielement e-commerce, webhooks et confirmation serveur
Notifications	Email, Brevo, webhook SMS	configuration `.env`	Envoi ou simulation de communications client
Tests backend	PHPUnit	configuration `api/`	Validation API et règles métier
Tests frontend	Playwright	configuration `frontend/`	Parcours utilisateur, roles et captures documentaires
Conteneurisation	Docker Compose	fichiers `infra/`	Lancement reproductible de la stack locale

4.2 Backend : PHP 8.2 et Symfony 6.4

Symfony est adapte a ce projet car il fournit :

- un routage clair
- une securite configurable
- une injection de dependances mature
- une integration solide avec Doctrine
- une structure naturelle en controllers, services, repositories et entites

PHP 8.2 reste pertinent pour une application de gestion car il est stable, largement supporté et très adapté aux architectures API classiques.

4.3 Frontend : React, TypeScript et Vite

React permet de construire des écrans riches comme la caisse, le planning ou le CRM.

TypeScript est utilisé pour :

- clarifier les payloads API
- limiter les erreurs de manipulation de données
- documenter les structures côté frontend

Vite a été choisi pour :

- sa rapidité de build
- une configuration légère
- une intégration simple avec React

4.4 Base de données : MySQL 8

MySQL est cohérent avec les besoins du projet :

- données fortement relationnelles
- nombreuses jointures
- historique des ventes, paiements, rendez-vous et mouvements de stock
- exécution simple dans Docker

4.5 Docker et verrouillage des versions

Docker Compose permet de reproduire le même environnement sur différents postes.

Les versions sont verrouillées pour limiter les régressions liées a des outils trop récents.

Versions de reference :

- Node.js `20.20.2`
- npm `10.8.2`
- Symfony `6.4`
- PHP runtime Docker `8.2`
- MySQL `8.0`

Fichiers de verrouillage :

- `.nvmrc`
- `.node-version`
- `frontend/package-lock.json`
- `api/composer.lock`
- `frontend/.npmrc`

5. Architecture generale

5.1 Vue d'ensemble

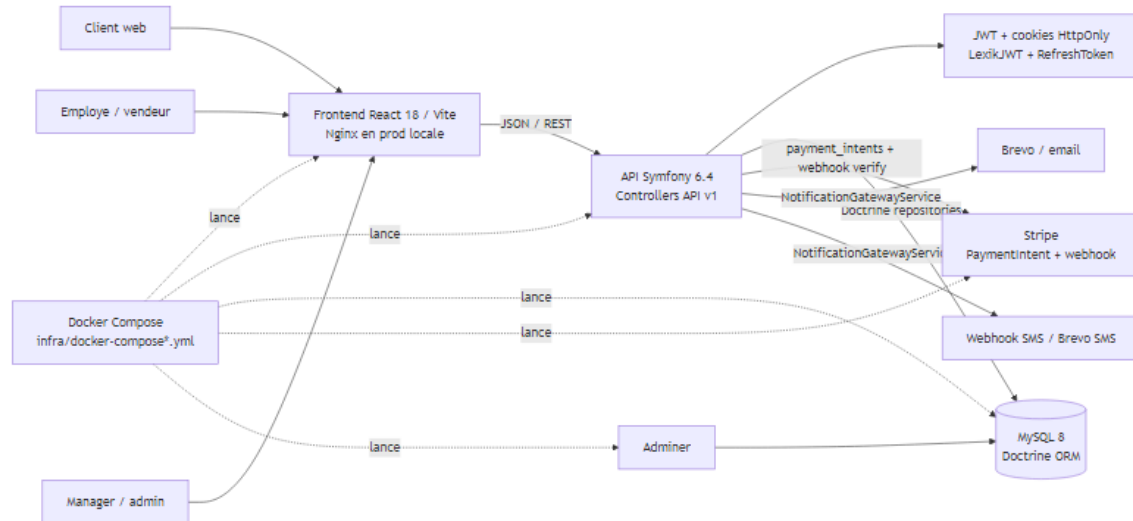
Procuratio suit une architecture web en trois blocs :

1. frontend React
2. API Symfony
3. base MySQL

Des services externes complètent l'application :

- Stripe pour le Paiement
- Brevo ou mail natif pour les emails
- webhook SMS selon configuration

Schema d'architecture globale :



5.2 Organisation du dépôt

```
app/  
|-- api/  
|-- frontend/  
|-- docs/  
|-- infra/  
|-- scripts/  
|-- .env  
|-- .env.example  
|-- .env.prod.example  
|-- .nvmrc  
|-- .node-version  
`-- README.md
```

5.3 Séparation des responsabilités

Le frontend gère :

- l'affichage
- l'experience utilisateur
- les appels API
- les guards de navigation
- les états locaux de formulaire

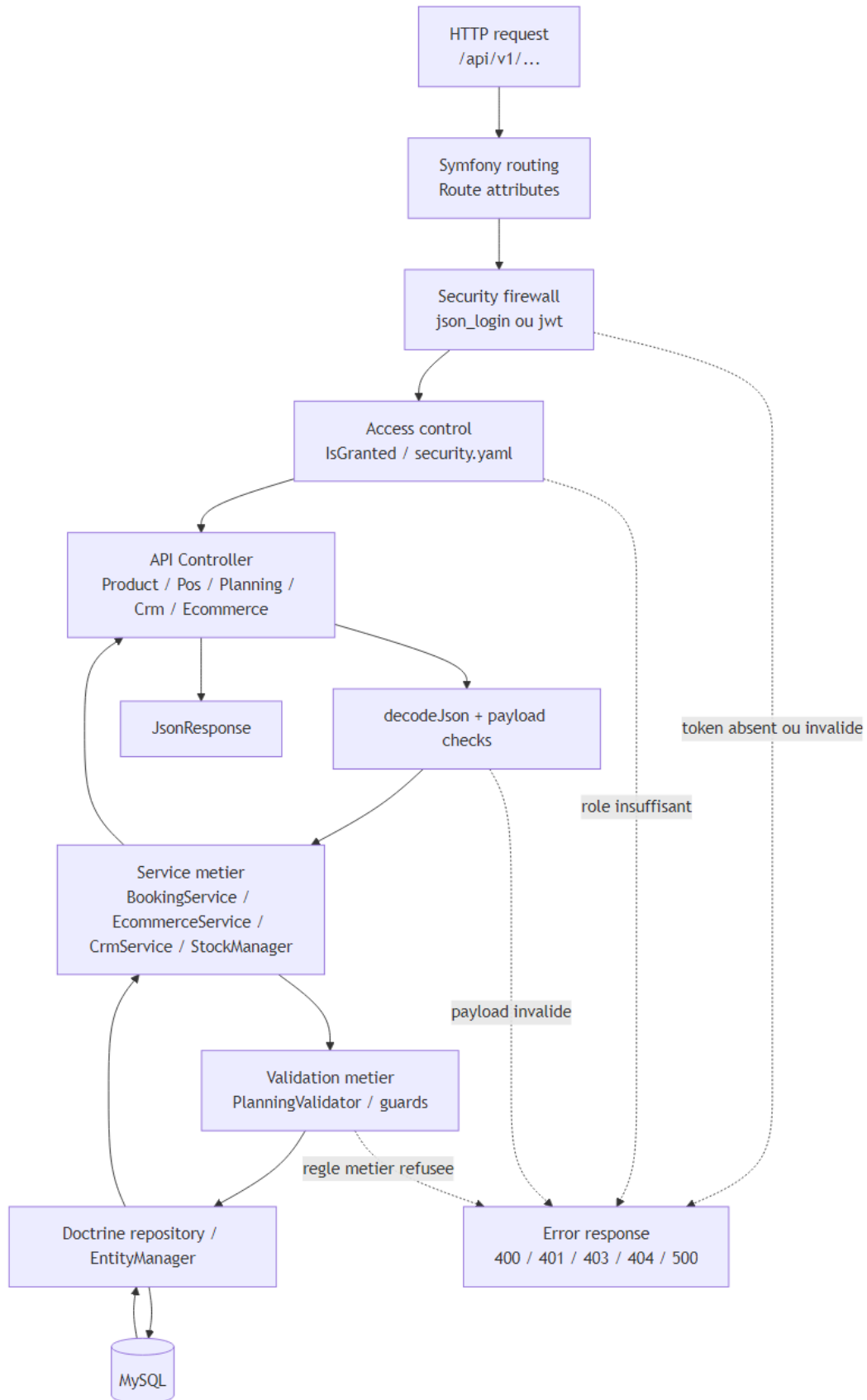
Le backend gère :

- l'authentification
- les permissions
- les validations serveur
- les règles métier
- la persistance
- les integrations externes sensibles

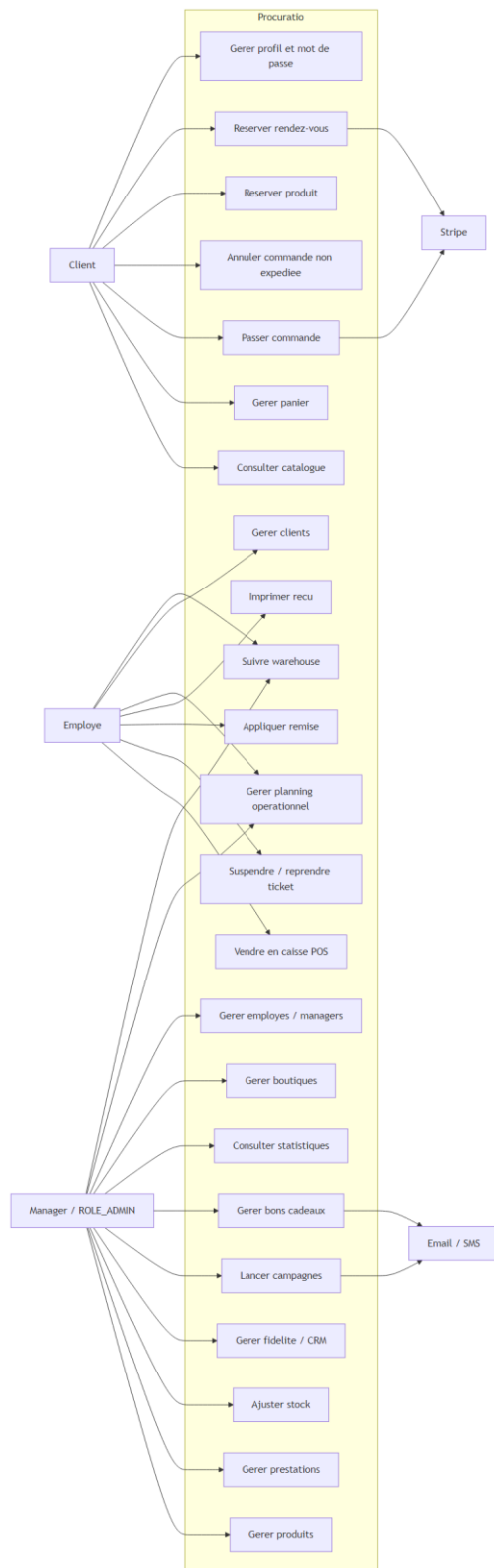
La base de données conserve :

- les données transactionnelles
- les historiques
- les relations métier

Schema de circulation interne:



5.4 Diagramme de cas d'utilisation



6. Architecture backend

6.1 Structure principale

```
api/  
|-- config/  
|-- public/  
|-- src/  
| |-- Command/  
| |-- Controller/  
| |-- DataFixtures/  
| |-- Demo/  
| |-- Entity/  
| |-- EventSubscriber/  
| |-- Repository/  
| |-- Security/  
| `-- Service/  
`-- tests/
```

6.2 Controllers principaux

L'API v1 est organisée par domaines :

- `ProductController`
- `ServiceController`
- `CatalogController`
- `PosController`
- `PlanningController`
- `CrmController`
- `BackofficeCustomerController`
- `EcommerceController`
- `WarehouseController`
- `StatsController`
- `EmployeeManagementController`
- `ManagerManagementController`
- `StoreController`
- `SystemController`

Chaque controller expose les routes d'un sous-domaine et délègue la logique importante a des services.

6.3 Services métier principaux

- `BookingService` : disponibilités, reservation et sessions de booking
- `PlanningValidator` : contraintes horaires, conflits et coherence planning
- `CrmService` : fidelite, campagnes, rappels et bons cadeaux
- `EcommerceService` : panier, commande, réservation produit et annulation
- `StockManager` : mouvements de stock et cohérence métier
- `SaleCalculator` : calculs POS, taxes et remises
- `StripeService` : paiement Stripe et webhook
- `NotificationGatewayService` : abstraction email / SMS
- `GiftVoucherDocumentService` : document de bon cadeau
- `StatsService` : indicateurs de dashboard

6.4 Gestion des erreurs

Le projet tend vers des messages runtime sobres et cohérents.

Les erreurs publiques doivent rester utiles sans exposer d'information sensible.

Principes:

- validation serveur avant traitement
- message utilisateur clair
- detail technique reserve aux logs
- pas d'exposition de secret, SQL ou stack trace

7. Architecture frontend

7.1 Structure principale

```
frontend/  
|-- public/  
|-- src/  
| |-- api/  
| |-- app/  
| |-- auth/  
| |-- hooks/  
| |-- layouts/  
| |-- pages/  
| |-- types/  
| |-- ui/  
| `-- utils/  
`-- tests/
```

7.2 Espaces applicatifs

Le routeur séparé :

- `/backoffice`` pour managers et employes
- `/client`` pour les clients
- les pages publiques de catalogue et reservation

Le backend reste la référence de sécurité.

Les guards frontend servent à guider l'utilisateur et à éviter les accès incohérents.

7.3 Pages structurantes

- ``ProductsPage``
- ``ServicesPage``
- ``PosPage``
- ``PlanningPage``
- ``CustomersPage``
- ``CrmPage``
- ``WarehousePage``

- `ProfilePage`
- `BackOfficeProfilePage`

7.4 Principes UI

L'interface est construite autour de composants réutilisables, de formulaires contrôles et de messages d'état explicites.

Les écrans denses ont été progressivement découpés pour réduire les composants monolithiques.

8. Architecture de la base de données

8.1 Vue d'ensemble

Le modèle est relationnel et organise par domaines.

Modèle conceptuel de données, vue simplifiée :

```

USER
|-- CUSTOMER
|   |-- CART
|   |-- ORDER
|   |-- APPOINTMENT
|   |-- LOYALTY_ACCOUNT
|   |-- GIFT_VOUCHER
|
|-- EMPLOYEE
    |-- STORE
    |-- EMPLOYEE_AVAILABILITY
    |-- APPOINTMENT

STORE
|-- BUSINESS_HOUR
|-- EMPLOYEE
|-- SALE
|-- APPOINTMENT
  
```

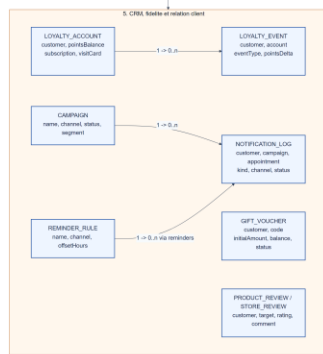
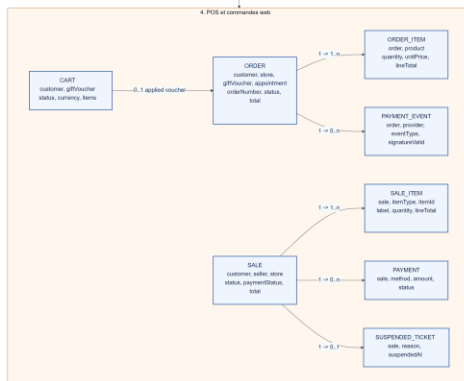
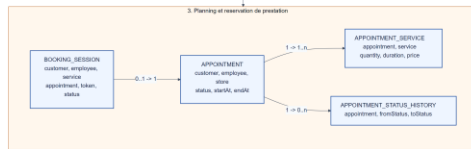
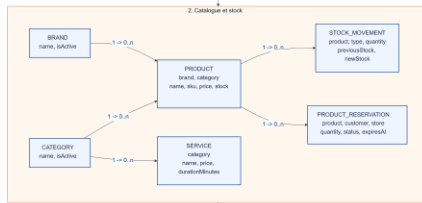
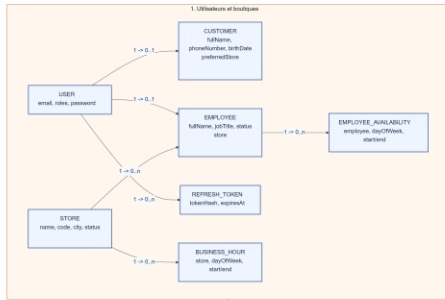
PRODUCT
|-- BRAND
|-- CATEGORY
|-- STOCK_MOVEMENT
`-- PRODUCT_RESERVATION

SERVICE
`-- CATEGORY

SALE
|-- SALE_ITEM
|-- PAYMENT
`-- PAYMENT_EVENT

ORDER
|-- ORDER_ITEM
`-- PAYMENT

Schema relationnel simplifie :



Relations transmissibles

- Importances
- Customer rôle partie, commentaires, votes, vendor-votes, fidèle, items cadeaux et avis
- Store rôle employé, boutiques, ventes, commandes, refresh-votes et reservations
- Product rôle stock, particularités items, reservations et avis
- Service rôle vendeur-votes et sessions de booking

users(id, email, password, roles, temporary_password_flag, created_at)
customers(id, user_id, first_name, last_name, email, phone, birth_date)
employees(id, user_id, store_id, first_name, last_name, role_label, active)
stores(id, name, city, address, active)

products(id, brand_id, category_id, name, price, quantity, active)
services(id, category_id, name, price, duration_minutes, active)
stock_movements(id, product_id, store_id, type, quantity, reason, created_at)

sales(id, customer_id, employee_id, store_id, status, total, paid_at)
sale_items(id, sale_id, product_id, service_id, quantity, unit_price, discount)
payments(id, sale_id, order_id, provider, status, amount, reference)
payment_events(id, payment_id, type, payload, created_at)

appointments(id, customer_id, employee_id, store_id, status, starts_at, ends_at)
appointment_services(id, appointment_id, service_id, quantity, price)
business_hours(id, store_id, weekday, opens_at, closes_at, is_open)
employee_availabilities(id, employee_id, weekday, starts_at, ends_at, is_available)

loyalty_accounts(id, customer_id, points_balance, membership_name, visit_goal, visits_used)
loyalty_events(id, loyalty_account_id, operation, points, reason, created_at)
gift_vouchers(id, customer_id, recipient_email, code, amount, expires_at, status)
orders(id, customer_id, status, total, payment_status, created_at)
order_items(id, order_id, product_id, quantity, unit_price)

8.2 Utilisateurs et profils

- `User`
- `Customer`
- `Employee`
- `Store`
- `RefreshToken`

Un `User` porte l'authentification.

Un client ou un employé est relié à cet utilisateur.

Un employé est rattaché à une boutique.

8.3 Catalogue et stock

- `Product`

- `Service`
- `Brand`
- `Category`
- `StockMovement`
- `ProductReservation`
- `ProductReview`

Les marques et catégories structurent le catalogue.

Les mouvements de stock assurent la traçabilité.

8.4 Vente et Paiement

- `Sale`
- `SaleItem`
- `SuspendedTicket`
- `Payment`
- `PaymentEvent`

Une vente contient plusieurs lignes.

Les paiements et événements permettent de tracer l'encaissement, l'annulation et les intégrations externes.

8.5 Rendez-vous et planning

- `Appointment`
- `AppointmentService`
- `AppointmentStatusHistory`
- `BusinessHour`
- `EmployeeAvailability`
- `BookingSession`

Les rendez-vous relient client, employé, boutique et prestations.

Les disponibilités employées doivent rester compatibles avec les horaires boutique.

8.6 CRM et fidelite

- `LoyaltyAccount`
- `LoyaltyEvent`
- `Campaign`
- `ReminderRule`
- `NotificationLog`
- `GiftVoucher`

Le CRM conserve l'historique des actions relationnelles et des avantages client.

8.7 E-commerce

- `Cart`
- `Order`
- `OrderItem`
- `StoreReview`

Les commandes conservent leurs lignes et leur statut.

Une commande client peut être annulée tant qu'elle n'est pas expédiée.

9. Conception de sécurité

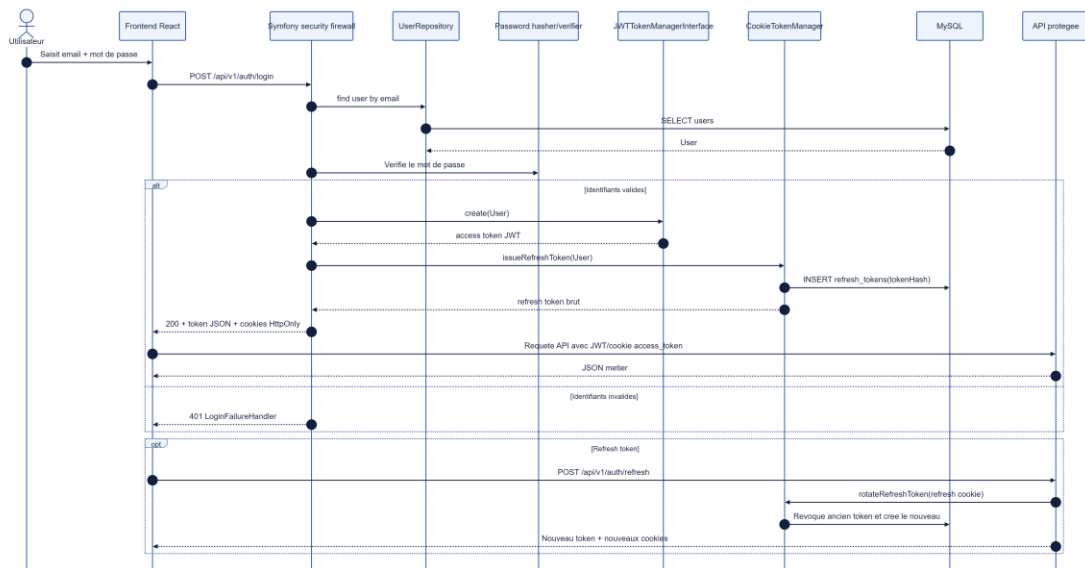
9.1 Authentification

L'API utilise une authentification JWT stateless.

Routes principales :

- `POST /api/v1/auth/login`
- `POST /api/v1/auth/refresh`
- `POST /api/v1/auth/logout`
- `GET /api/v1/me`

Sequence d'authentification JWT:



9.2 Autorisation

La hiérarchie de rôles est définie dans `security.yaml` :

- `ROLE\_CUSTOMER`
- `ROLE\_EMPLOYEE`
- `ROLE\_ADMIN`

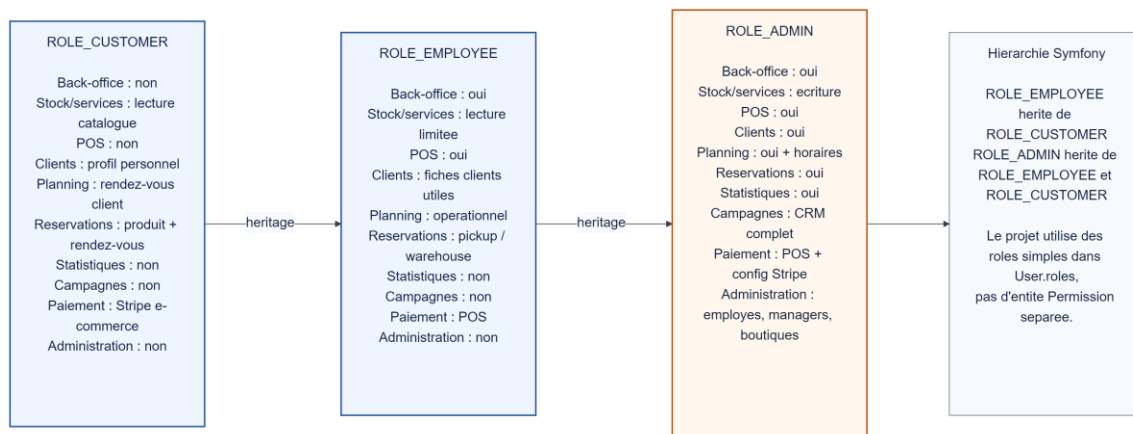
Les routes publiques sont limitées aux besoins explicites :

- healthcheck

- login
- catalogue public
- disponibilités publiques
- documentation OpenAPI
- webhook Stripe

Le reste de l'API exige un utilisateur authentifié.

Synthèse des rôles et permissions :



9.3 Gestion des mots de passe

Chaque utilisateur peut modifier son mot de passe depuis son profil.

Lorsqu'un mot de passe temporaire est généré par un manager, l'utilisateur est incité à le remplacer.

9.4 Secrets et variables sensibles

Les secrets ne doivent pas être versionnés.

Variables sensibles principales :

- `APP\_SECRET`
- `JWT\_PASSPHRASE`
- `STRIPE\_SECRET\_KEY`
- `STRIPE\_WEBHOOK\_SECRET`
- `BREVO\_API\_KEY`
- `SMS\_WEBHOOK\_TOKEN`

9.5 Paiement et webhooks

Le webhook Stripe est public cote JWT mais protège par vérification de signature.

Cette séparation est volontaire : Stripe ne peut pas fournir un JWT applicatif, la confiance repose donc sur le secret webhook.

9.6 Données exposées

Les réponses API doivent exposer seulement les données utiles à l'écran.

Les messages d'erreur doivent éviter les détails techniques exploitables.

10. API et contrats d'échange

10.1 Conventions générales

L'API est exposée sous `/api/v1`.

Les échanges utilisent JSON.

Les routes de lecture utilisent principalement `GET`, les créations `POST`, les modifications `PUT` ou `PATCH`, les suppressions `DELETE`.

10.2 Routes de base

GET /api/v1/health : vérifie que l'API répond correctement.

POST /api/v1/auth/login : connecte un utilisateur et retourne la session/JWT.

GET /api/v1/me : retourne les informations du compte connecté.

GET /api/v1/admin/ping : vérifie qu'un compte admin a bien accès aux routes protégées.

10.3 Catalogue et stock

GET /api/v1/products : liste les produits avec filtres éventuels.

POST /api/v1/products : crée un nouveau produit.

PUT /api/v1/products/{id} : modifie un produit existant.

DELETE /api/v1/products/{id} : supprime ou bloque la suppression d'un produit selon ses usages.

POST /api/v1/products/{id}/stock-adjustments : ajuste le stock d'un produit et trace le mouvement.

GET /api/v1/services : liste les prestations disponibles.

POST /api/v1/services : crée une nouvelle prestation.

PUT /api/v1/services/{id} : modifie une prestation existante.

DELETE /api/v1/services/{id} : supprime ou bloque la suppression d'une prestation selon ses usages.

GET /api/v1/catalog/brands : liste les marques du catalogue.

GET /api/v1/catalog/categories : liste les catégories/types du catalogue.

10.4 E-commerce

GET /api/v1/catalog/products : liste les produits visibles côté client.

GET /api/v1/catalog/products/{id} : retourne le détail d'un produit client.

GET /api/v1/cart : récupère le panier courant du client.

POST /api/v1/cart/items : ajoute un produit au panier.

PUT /api/v1/cart/items/{productId} : modifie la quantité d'un produit du panier.

DELETE /api/v1/cart/items/{productId} : retire un produit du panier.

POST /api/v1/checkout : transforme le panier en commande et prépare le paiement.

GET /api/v1/orders/me : liste les commandes du client connecté.

GET /api/v1/orders/{orderNumber} : affiche le détail d'une commande.

POST /api/v1/payments/stripe/webhook : reçoit les événements Stripe et met à jour le statut de paiement.

10.5 Fidélité et bons cadeaux

GET /api/v1/loyalty/me : retourne le solde fidélité et les avantages du client.

GET /api/v1/crm/gift-vouchers/{id}/print : génère le document imprimable d'un bon cadeau.

POST /api/v1/crm/gift-vouchers/{id}/send : envoie un bon cadeau par email si la configuration le permet.

10.6 Réservation produit

POST /api/v1/catalog/products/{id}/reservations : réserve un produit pour un client.

GET /api/v1/reservations/me : liste les réservations produit du client connecté.

POST /api/v1/reservations/{id}/cancel : annule une réservation produit active.

POST /api/v1/reservations/{id}/picked-up : marque une réservation produit comme récupérée en boutique.

10.7 Documentation OpenAPI

La documentation API est disponible localement :

- dev : `http://localhost:48180/api/doc`
- prod locale : `http://localhost:48280/api/doc`

11. Outils de vente et de réservation

11.1 Caisse / POS

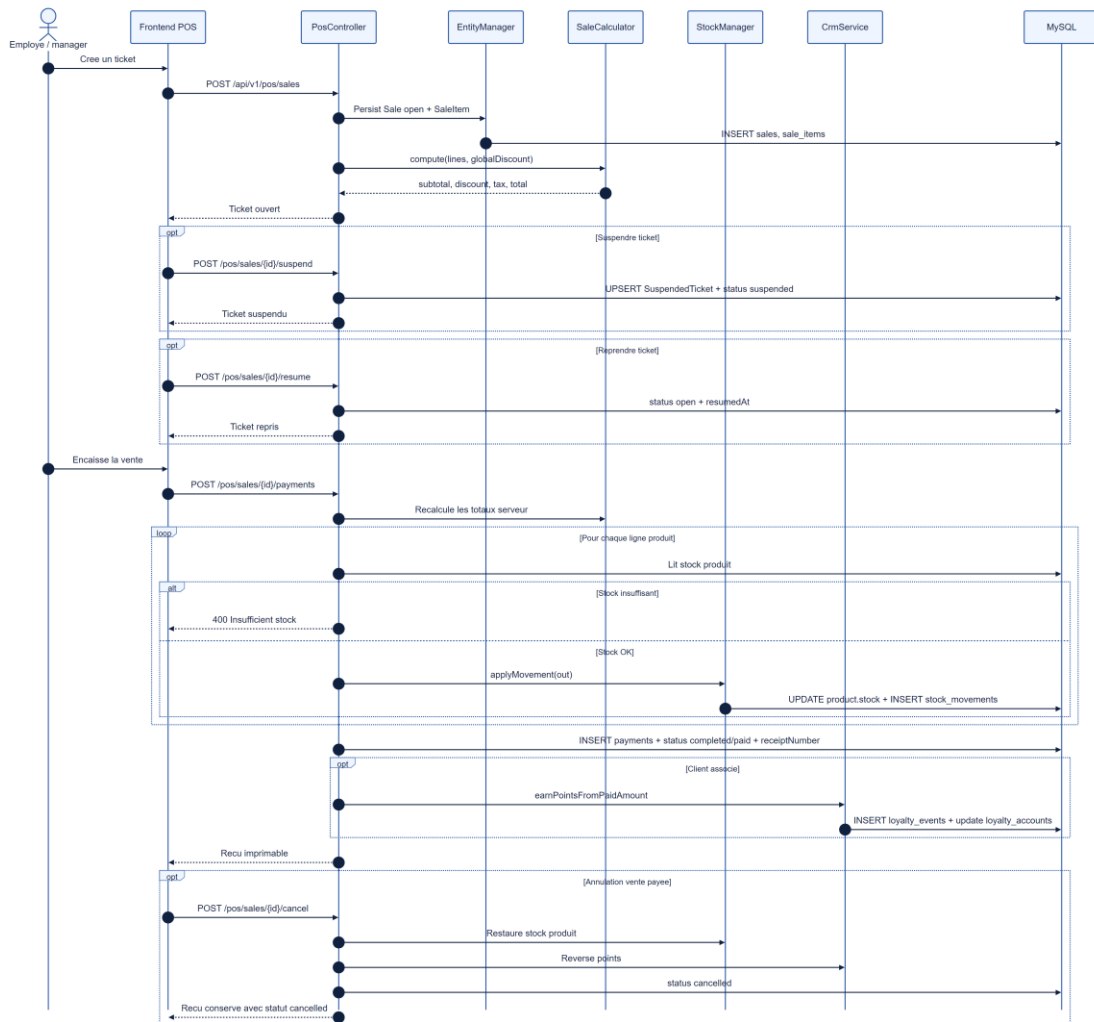
Le module POS permet :

- creation de ticket
- ajout de produits et services
- remises de ligne
- remise globale
- suspension et reprise
- encaissement
- impression du reçu
- annulation d'une vente encaissée

Une vente annulée ne peut pas être suspendue ni reprise.

Elle reste consultable pour l'historique et l'impression du reçu.

Séquence de vente en caisse :



11.2 Planning

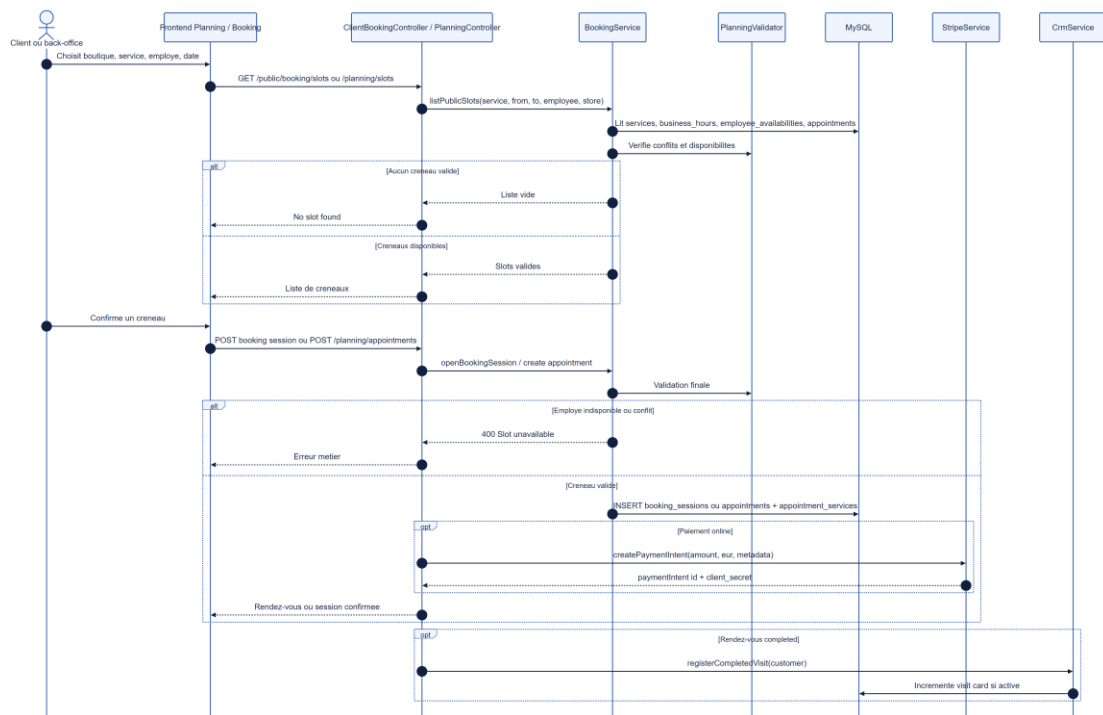
Le planning permet :

- vues jour, semaine, mois et année
- creation manuelle de rendez-vous
- recherche de créneaux disponibles
- gestion des horaires salon
- résumé des horaires pour les employés
- complétion ou annulation d'un rendez-vous

Les employés visibles sont filtrés par boutique.

Les dates et horaires disponibles respectent les horaires d'ouverture et les disponibilités employés.

Sequence de reservation :

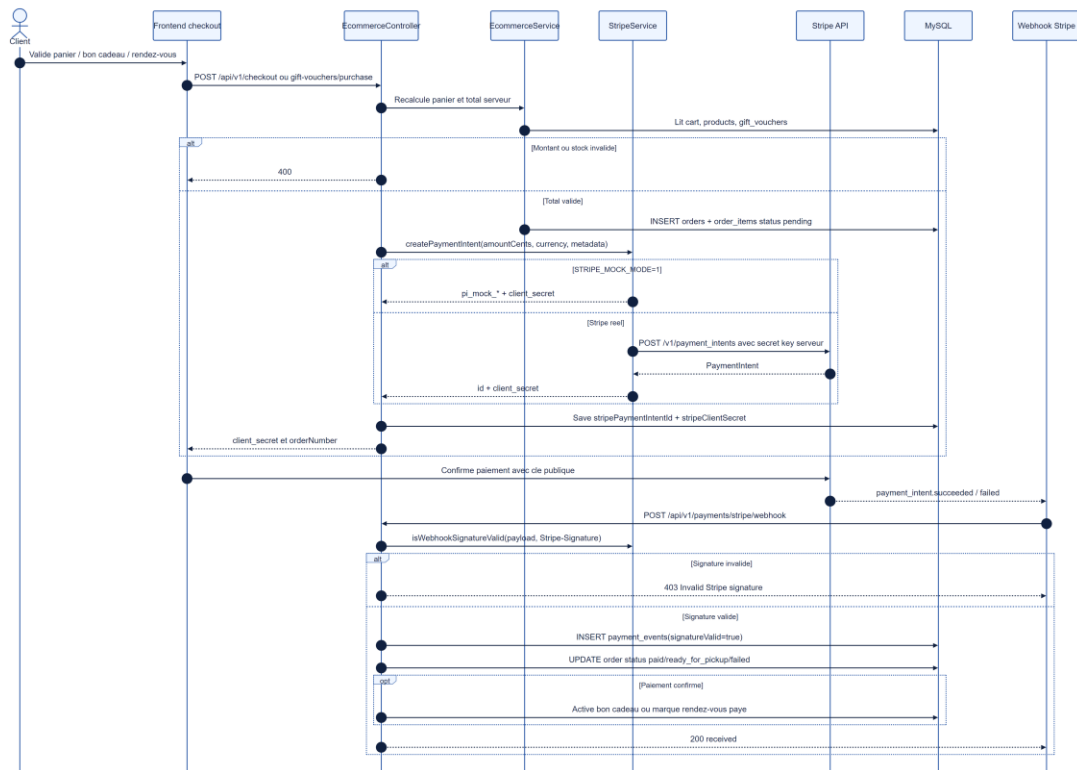


11.3 E-commerce

Le parcours e-commerce permet :

- consultation du catalogue
- panier
- checkout
- paiement Stripe
- consultation des commandes
- annulation tant que la commande n'est pas expédiée

Flux de paiement sécurisé :



12. CRM, fidélité et relation client

12.1 Points de fidélité

Le CRM permet d'ajouter ou consommer des points de fidélité.

Les actions peuvent être appliquées individuellement ou en masse sur des clients sélectionnés.

12.2 Membership et cartes de visites

Le membership représente le programme ou abonnement de fidélité du client.

La carte de visites suit une progression de type `visites utilisées / objectif`.

Une carte s'incrémente lorsqu'un rendez-vous éligible passe en statut `completed`.

12.3 Campagnes

Les campagnes permettent de préparer des communications email ou SMS.

Les logs de notification permettent de tracer les envois ou tentatives.

12.4 Bons cadeaux

Les bons cadeaux peuvent être créés depuis le CRM.

Ils peuvent être associés à un client ou à une adresse email.

Les bons non expirés associés à l'adresse du client sont visibles depuis son profil.

13. Paiement, notifications et intégrations externes

13.1 Stripe

Stripe est utilisé pour le paiement e-commerce.

Le backend crée ou valide les paiements et traite les webhooks.

Variables importantes :

- `STRIPE\_SECRET\_KEY`
- `STRIPE\_WEBHOOK\_SECRET`
- `STRIPE MOCK\_MODE`

13.2 Emails

Le projet peut utiliser un mode mail local ou Brevo selon configuration.

Variables importantes :

- `NOTIFICATIONS\_EMAIL\_MODE`
- `NOTIFICATIONS\_MAIL\_FROM`
- `NOTIFICATIONS\_MAIL\_REPLY\_TO`
- `BREVO\_API\_KEY`

13.3 SMS

Les SMS peuvent être délégués a un webhook externe.

Variables importantes :

- `NOTIFICATIONS\_SMS\_MODE`
- `SMS\_SENDER`
- `SMS\_WEBHOOK\_URL`
- `SMS\_WEBHOOK\_TOKEN`

14. Infrastructure et déploiement local

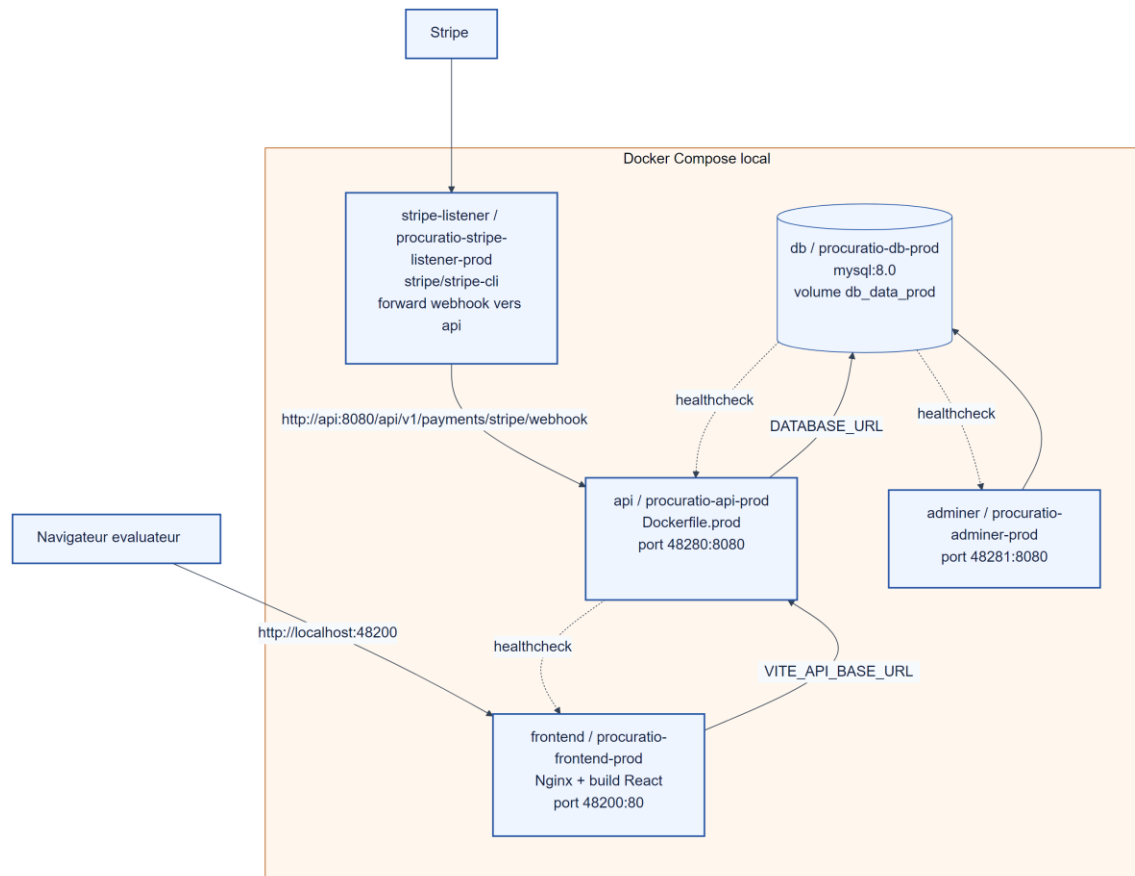
14.1 Stack Docker

La stack locale comprend :

- MySQL
- API Symfony
- frontend Nginx
- Adminer

- Stripe CLI listener selon configuration

Diagramme de déploiement local :



14.2 Ports utiles

Environnement développement :

- frontend : ``http://localhost:43100``
- API : ``http://localhost:48180``
- OpenAPI : ``http://localhost:48180/api/doc``
- Adminer : ``http://localhost:48181``
- MySQL : ``localhost:43306``

Environnement prod locale:

- frontend : `http://localhost:48200`
- API : `http://localhost:48280`
- OpenAPI : `http://localhost:48280/api/doc`
- Adminer : `http://localhost:48281`

14.3 Démarrage

Développement :

```
docker compose --env-file .env -f infra/docker-compose.yml up -d --build
```

Prod locale :

```
docker compose --env-file .env -f infra/docker-compose.prod.yml up -d --build
```

Arret :

```
docker compose --env-file .env -f infra/docker-compose.yml down  
docker compose --env-file .env -f infra/docker-compose.prod.yml down
```

15. Prise en main développeur

15.1 Installation pas a pas

Deux modes de récupération du projet sont possibles.

Cas 1 : le projet est récupéré avec Git.

1. Cloner le dépôt : <https://github.com/jvaldy/Procuratio.git> .
2. Se placer dans le dossier racine de l'application.
3. Copier `.env.example` vers `.env`.
4. Renseigner les variables sensibles.
5. Lancer la stack Docker.
6. Vérifier l'API doc.
7. Se connecter avec un compte de test local.

Cas 2 : le projet est fourni sous forme d'archive ZIP ou de dossier source.

1. Extraire l'archive dans un dossier local sans espace ni caractère spécial dans le chemin si possible.
2. Vérifier que les dossiers ``api/``, ``frontend/``, ``infra/``, ``docs/`` et ``scripts/`` sont présents.
3. Vérifier que les fichiers de verrouillage sont inclus : ``api/composer.lock`` et ``frontend/package-lock.json``.
4. Copier `.env.example` vers `.env` si le fichier `.env` n'est pas fourni.
5. Ajuster les variables locales : ports, secrets, Stripe, notifications.
6. Lancer Docker Desktop.
7. Exécuter la commande de démarrage Docker Compose adaptée.
8. Ouvrir le frontend et la documentation API pour valider l'installation.

Cette approche ne suppose pas que l'examineur ait accès à l'historique Git.

Le code source, les lockfiles et les fichiers d'environnement exemple suffisent pour lancer le projet.

15.2 Comptes de test locaux

- `admin@procuratio.local` / `Admin123!`
- `employee@procuratio.local` / `Employee123!`
- `customer@procuratio.local` / `Customer123!`

Ces comptes sont destinés a la démonstration locale.

Ils ne doivent pas être utilisés en production réelle.

15.3 Commandes de maintenance

```
docker compose --env-file .env -f infra/docker-compose.prod.yml ps
docker compose --env-file .env -f infra/docker-compose.prod.yml logs -f api
docker compose --env-file .env -f infra/docker-compose.prod.yml logs -f frontend
docker exec procuratio-api-prod php bin/console doctrine:migrations:migrate --no-
interaction
docker exec procuratio-api-prod php bin/console cache:clear
docker exec procuratio-api-prod php bin/console app:cleanup-e2e-data --no-interaction
```

15.4 Variables d'environnement critiques

- `DATABASE\_URL`
- `APP\_SECRET`
- `JWT\_PASSPHRASE`
- `CORS\_ALLOW\_ORIGIN`
- `STRIPE\_SECRET\_KEY`
- `STRIPE\_WEBHOOK\_SECRET`
- `NOTIFICATIONS\_MAIL\_FROM`
- `BREVO\_API\_KEY`

16. Tests et stratégie de validation

16.1 Stratégie globale

La stratégie de test combine :

- tests backend pour les règles API et métier
- tests Playwright pour les parcours utilisateur
- captures Playwright pour la documentation visuelle
- vérification manuelle des parcours critiques avant livraison

16.2 Backend

```
cd api  
php bin/phpunit
```

Zones critiques:

- authentification
- planning
- booking
- POS
- e-commerce
- permissions

16.3 Frontend

```
cd frontend  
npm run build  
npx playwright test
```

Parcours couverts :

- login
- annuaire admin
- planning manager
- POS
- e-commerce
- warehouse

- profile
- CRM

16.4 Captures documentaires

Un scenario Playwright génère des captures complètes des pages et modales par rôle.

Les captures sont destinées a la documentation utilisateur et a la recette visuelle.

17. Difficultés connues et solutions

17.1 Versions locales plus récentes

Si une machine utilise une version plus récente de Node ou npm, le build peut diverger.

La solution recommandée est d'utiliser les versions verrouillées ou le build Docker.

Commandes de vérification :

```
node -v  
npm -v  
docker --version  
docker compose version
```

17.2 Ports deja utilises

Si un port est deja pris, Docker peut refuser de lancer un conteneur.

Il faut arrêter le service concurrent ou modifier le port dans `.env`.

17.3 Emails non recus

Un email peut être préparé côté application sans être délivré par le provider.

Il faut vérifier:

- mode de notification
- clé Brevo
- domaine expéditeur
- logs transactionnels Brevo
- quotas et blocages provider

17.4 Données E2E visibles en prod locale

Les tests doivent nettoyer leurs données.

Si d'anciens reliquats persistent, une commande de nettoyage dédiée peut être lancée.

17.5 Taille des chunks frontend

Un warning vite sur la taille des chunks n'est pas bloquant.

C'est une piste d'optimisation, pas une erreur de build.

18. Limites et perspectives

18.1 Limites actuelles

- certains écrans frontend restent denses
- la délivrabilité email dépend d'un provider externe
- la documentation utilisateur doit être finalisée séparément
- certains messages applicatifs peuvent encore être harmonisés

18.2 Perspectives fonctionnelles

- enrichir les scenarios CRM avancés
- renforcer les statistiques magasin
- ajouter des exports plus complets
- ameliorer les outils de suivi de livraison email

18.3 Perspectives techniques

- continuer la modularisation des gros écrans
- renforcer les tests négatifs
- compléter l'OpenAPI avec davantage d'exemples de payloads

19. Annexes techniques

19.1 Documentation disponible

- `README.md` : lancement, commandes utiles et dépannage
- `docs/ARCHITECTURE.md` : synthese d'architecture
- `docs/TECHNICAL\_DOCUMENTATION.docx` : version Word de cette documentation
- `docs/captures/playwright/` : captures documentaires generees

19.2 Scripts utiles

Les scripts fournis ne sont pas obligatoires : ils encapsulent les commandes Docker les plus fréquentes afin d'éviter les erreurs de saisie.

Ils existent en version shell pour Linux/macOS et en version PowerShell pour Windows.

`bootstrap-env` prépare les fichiers d'environnement locaux a partir des modèles.

Il est utile lors d'une première installation ou lorsqu'un examinateur reçoit uniquement le code source.

```
sh scripts/bootstrap-env.sh  
powershell -ExecutionPolicy Bypass -File scripts/bootstrap-env.ps1
```

`stack-prod-up` lance la stack de démonstration locale avec les fichiers de configuration production locale.

Il reconstruit les images si nécessaire et démarre les services attendus.

```
sh scripts/stack-prod-up.sh  
powershell -ExecutionPolicy Bypass -File scripts/stack-prod-up.ps1
```

`stack-prod-down` arrête proprement les services de la stack locale.

Il est utile avant de libérer les ports ou de fermer l'environnement de démonstration.

```
sh scripts/stack-prod-down.sh  
powershell -ExecutionPolicy Bypass -File scripts/stack-prod-down.ps1
```

Les commandes Docker équivalentes restent documentées dans les sections d'installation et de maintenance.

20. Conclusion

Procuratio repose sur une architecture volontairement classique et maintenable :

- API Symfony modulaire
- frontend React type
- base relationnelle MySQL
- infrastructure Docker reproductible
- intégration Stripe pour les paiements
- notifications configurables
- tests backend et frontend sur les parcours essentiels

Le projet est structure pour couvrir un besoin métier concret : faire cohabiter vente, réservation, CRM, fidélité, e-commerce et suivi client dans un même outil.

Les choix techniques privilégient la stabilité, la compréhension par une équipe et la possibilité d'évoluer sans refonte majeure.